

```
    } else if (n % 2 === 0) {
        // RECURSIVE CASE (When n is even.)
        result = exponentByRecursion(a, n / 2);
        return result * result;
    } else if (n % 2 === 1) {
        // RECURSIVE CASE (When n is odd.)
        result = exponentByRecursion(a, Math.floor(n / 2));
        return result * result * a;
    }
}

document.write(exponentByRecursion(3, 6));
document.write(exponentByRecursion(10, 3));
document.write(exponentByRecursion(17, 10));
</script>
```

When you run this code, the output is identical to the iterative version:

```
729
1000
2015993900449
```

Each recursive call effectively cuts the problem size in half. This is what makes our recursive exponent algorithm faster than the iterative version; calculating 3^{1000} iteratively entails 1,000 multiplication operations, while doing it recursively requires only 23 multiplications and divisions. When running the Python code under a performance profiler, calculating 3^{1000} iteratively 100,000 times takes 10.633 seconds, but the recursive calculation takes only 0.406 seconds. That is a huge improvement!

Creating an Iterative Exponents Function Based on Recursive Insights

Our original iterative exponents function took a straightforward approach: loop the same number of times as the exponent power. However, this doesn't scale well for larger powers. Our recursive implementation forced us to think about how to break this problem into smaller subproblems. This approach turns out to be much more efficient.

Because every recursive algorithm has an equivalent iterative algorithm, we could make a new iterative exponents function based on the power rule